Паттерны Микросервисов

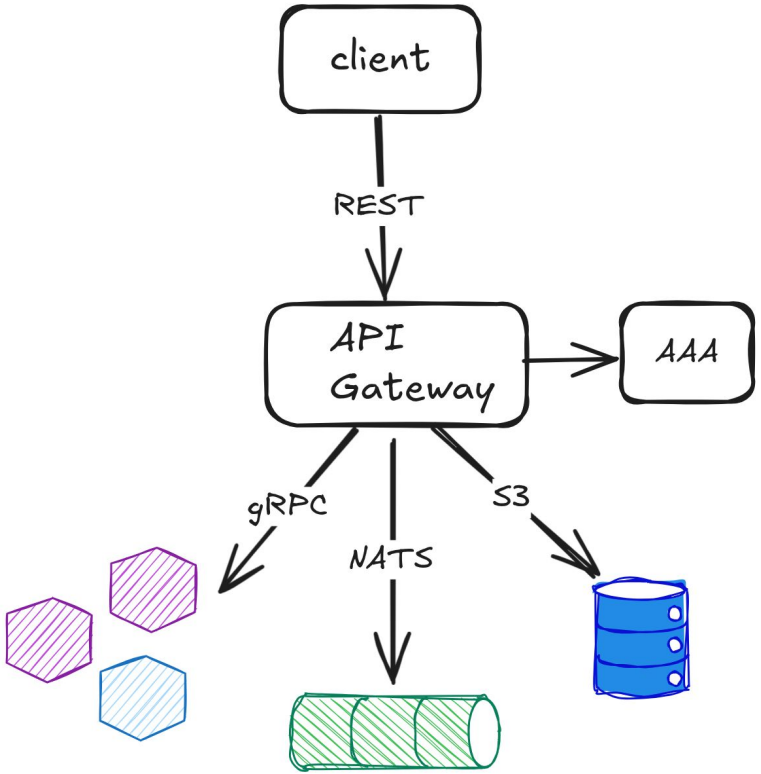
YADRO Go Course 2026/1

Паттерны микросервисов решают проблемы

..микросервисов 😛

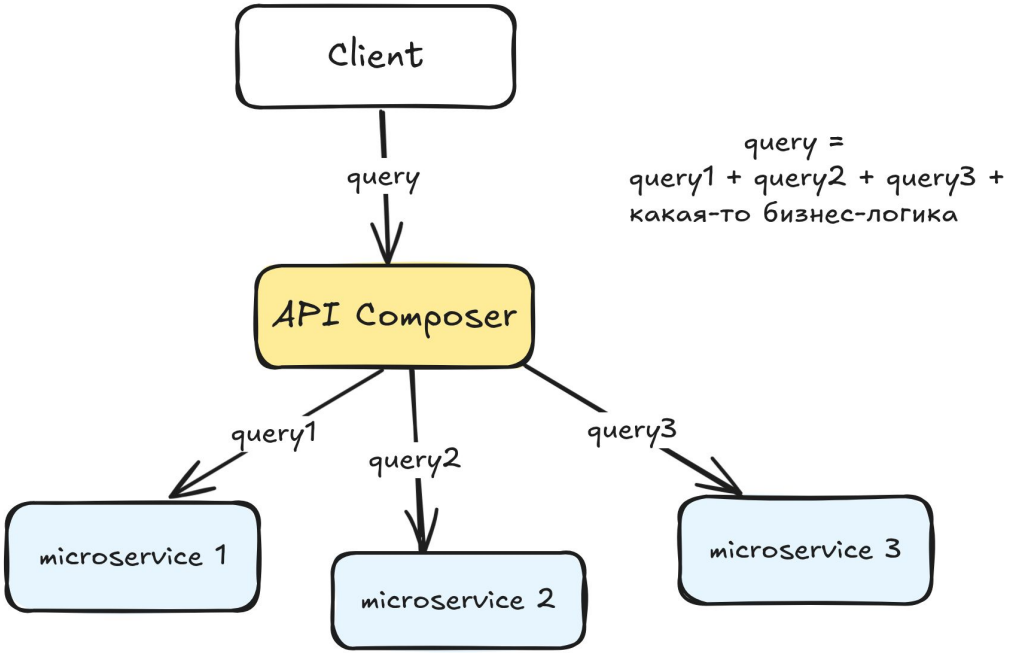
API Gateway

- Единая точка входа для клиентов
- Скрывает сложность системы
- Маршрутизация, балансировка
- Аутентификация и авторизация запросов, ограничение скорости
- Преобразование протоколов и данных
- Кэширование
- Мониторинг



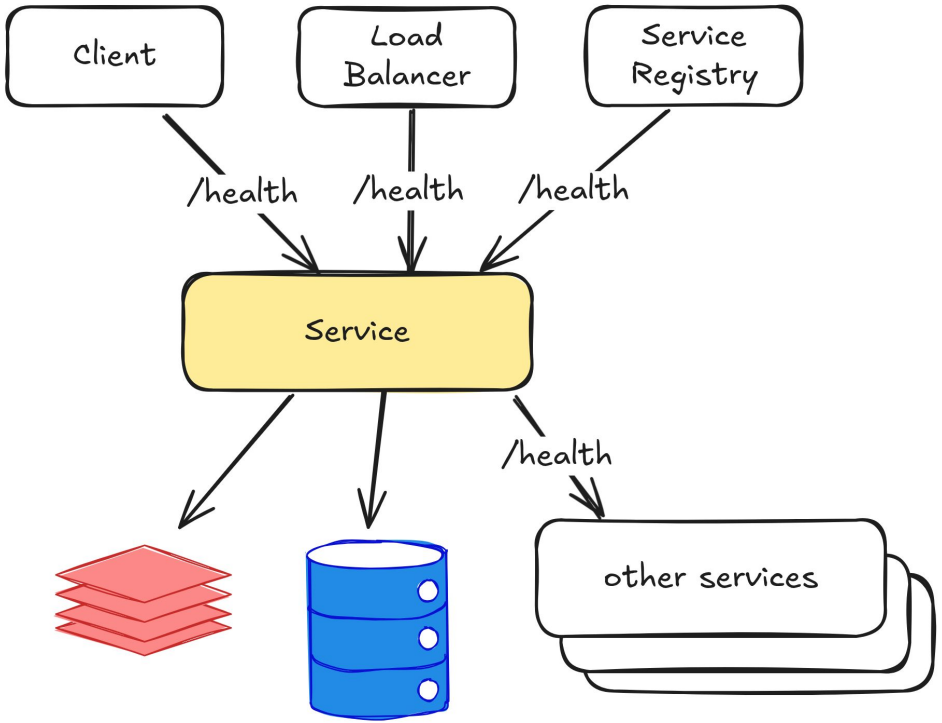
API Composer

- Объединяет несколько API
- Упрощает логику для клиента



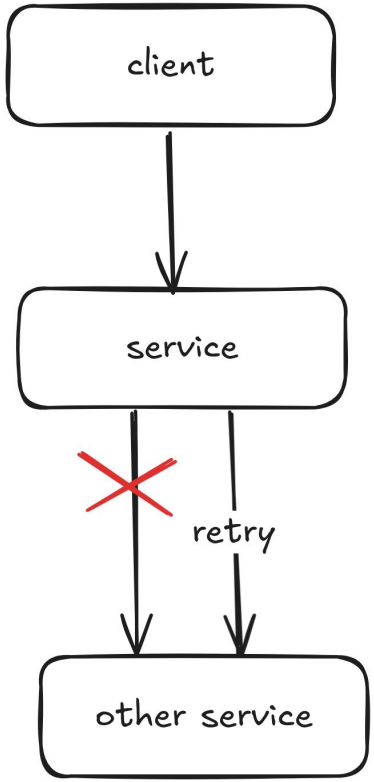
Health Check

- Раннее обнаружение проблем
- Автоматическое восстановление
- Балансировка нагрузки
- Типы:
  - Liveness (запущен?)
  - Readiness (готов?)
  - Deap Health Check (состояние компонентов)



Retry

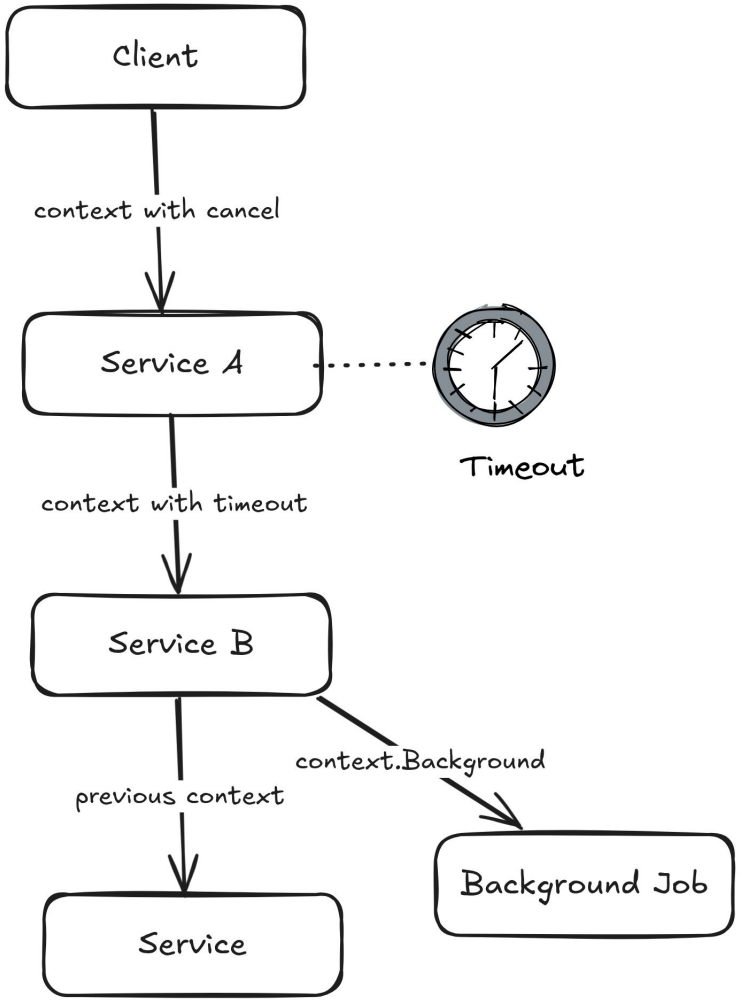
- Повышение устойчивости к сбоям
- Настраиваемые политики
  - список ошибок для повтора
  - начальная задержка
  - максимальная задержка
  - алгоритм увеличения задержки



Timeout

- Не ждем вечно
- Ограничение максимального времени выполнения операции
- Предотвращение зависаний и истощения ресурсов системы

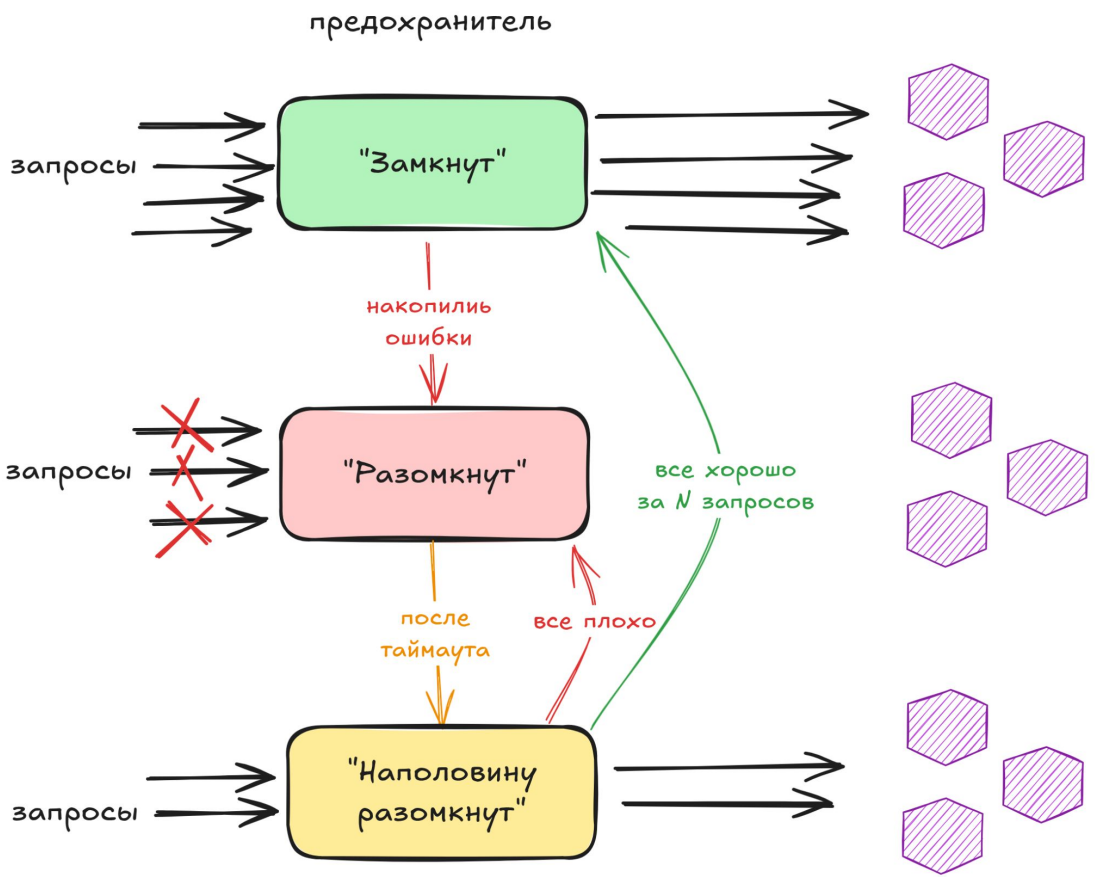
Настраиваем и проверяем контексты



Circuit Breaker

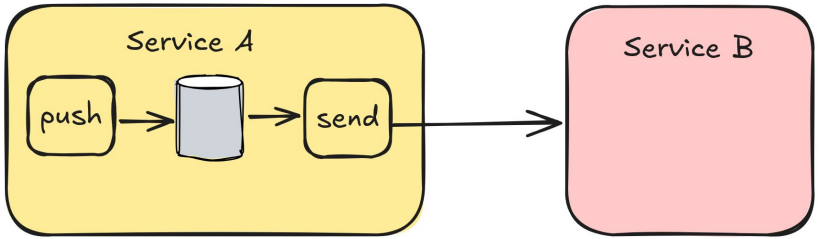
- Предотвращение каскадных сбоев
- Экономия ресурсов
- Время на восстановление
- Плавная деградация
- Раннее обнаружение проблем

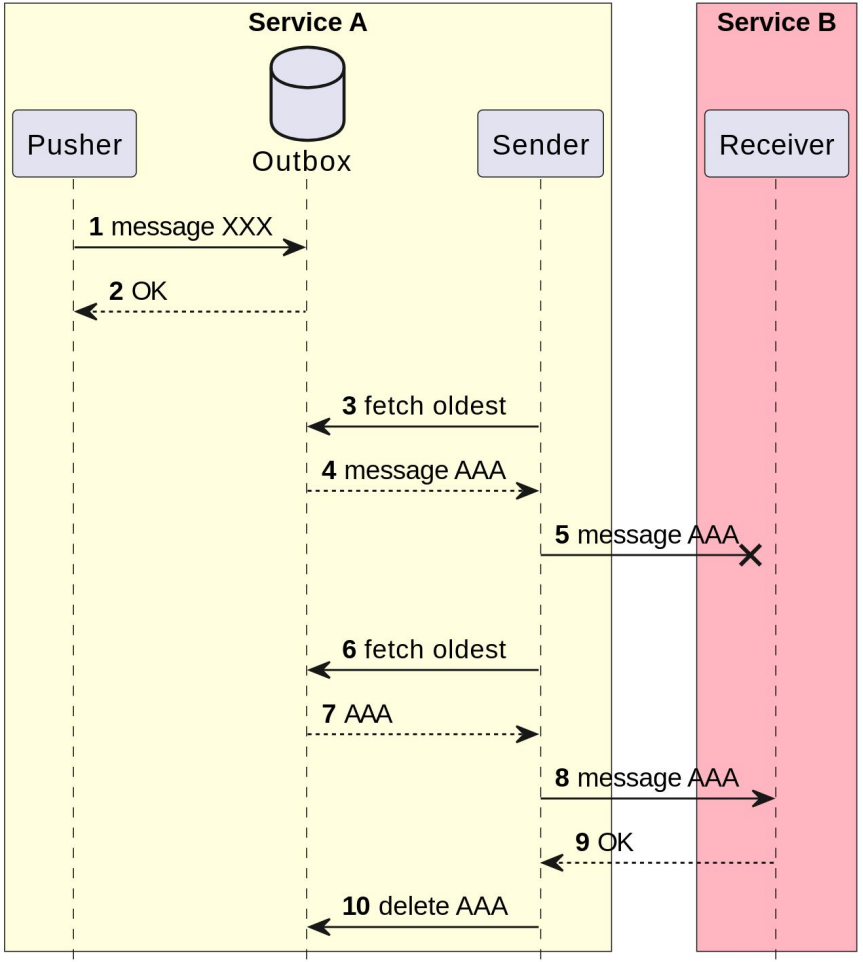




Transactional Outbox

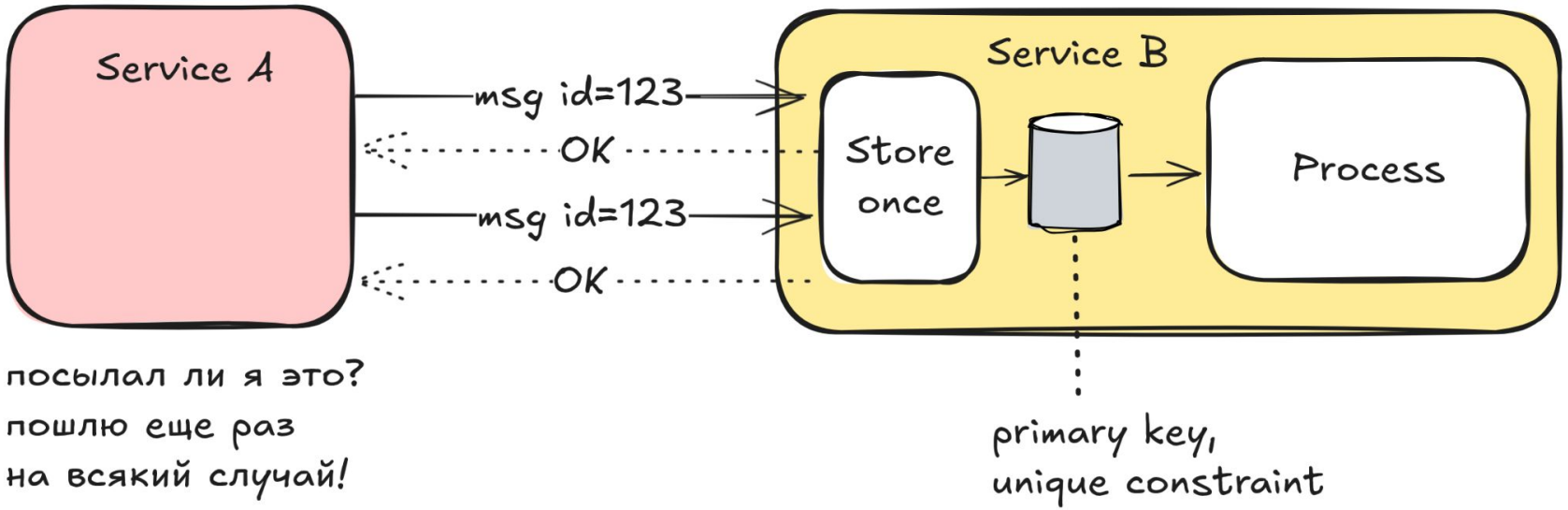
Как удостовериться, что сообщение послано?





Idempotent Receiver

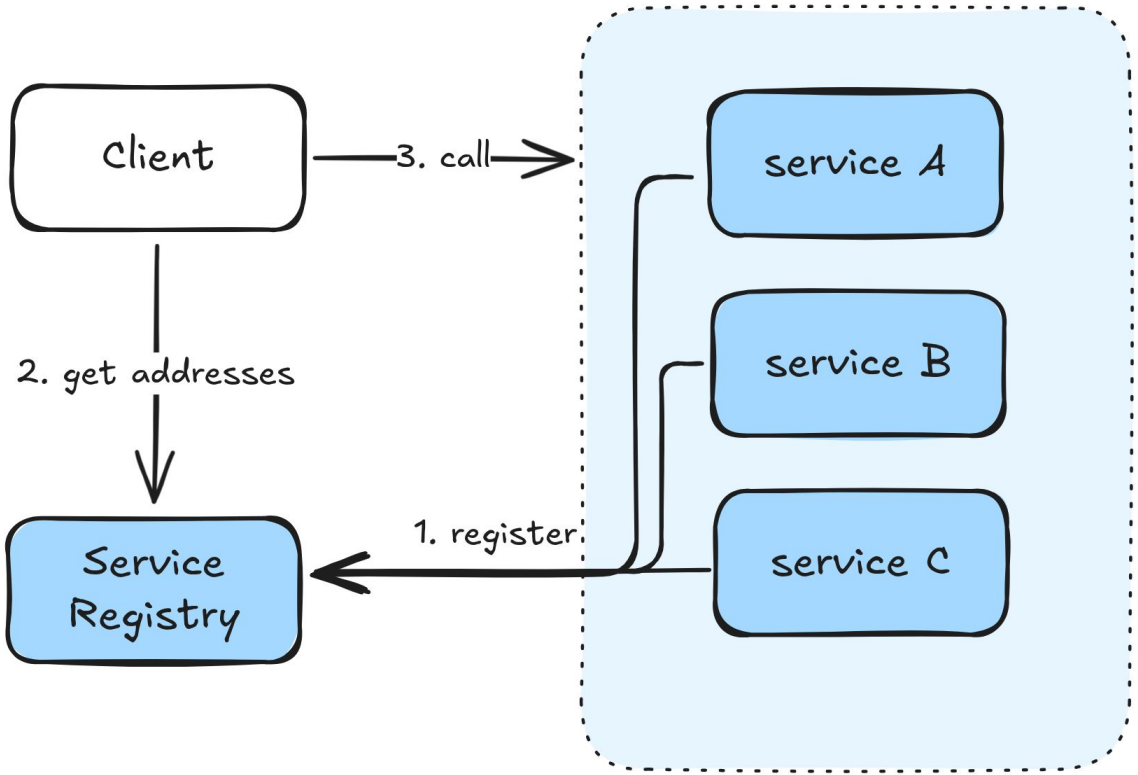
Как удостовериться, что уже не обработали сообщение?



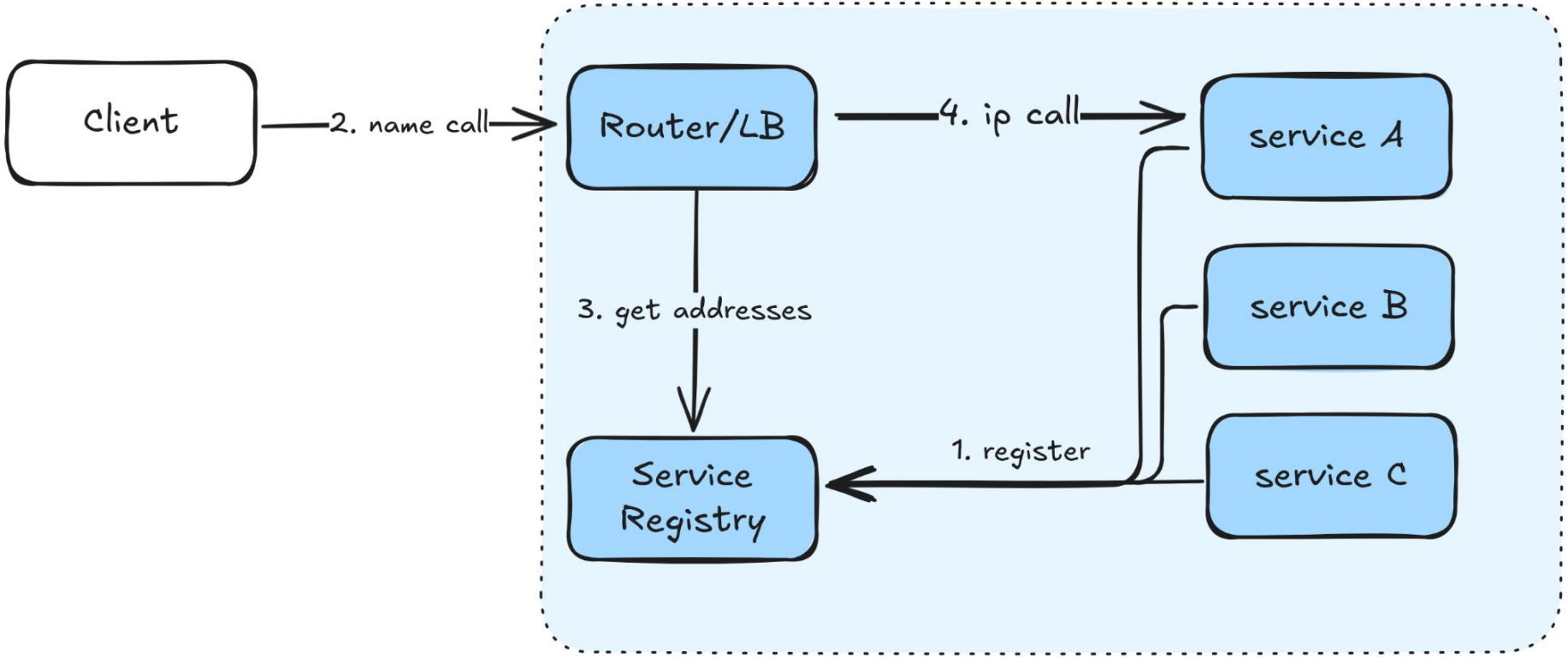
Service Discovery

- Определение местонахождения нужных клиенту сервисов
- Количество экземпляров сервисов и их расположение изменяются динамически
- Бывает client-side и server-side

Client-Side Service Discovery



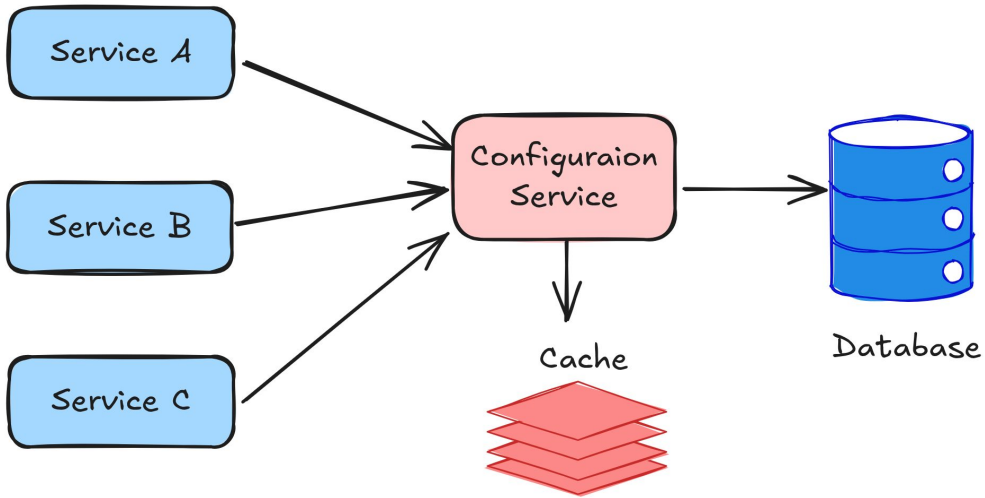
Server-Side Service Discovery



External Configuration

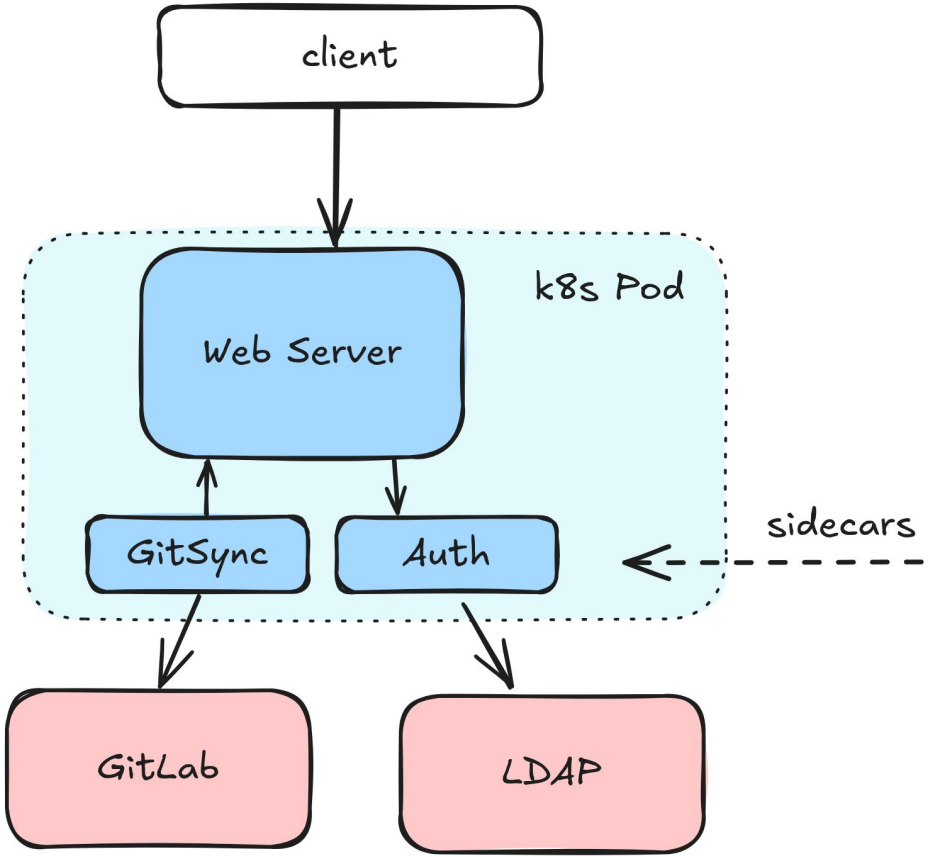
Хранение конфигурационных параметров приложения вне его кодовой базы

- Безопасность
- Гибкость развертывания
- Отслеживаемость изменений
- Динамическое обновление
- Централизованное управление



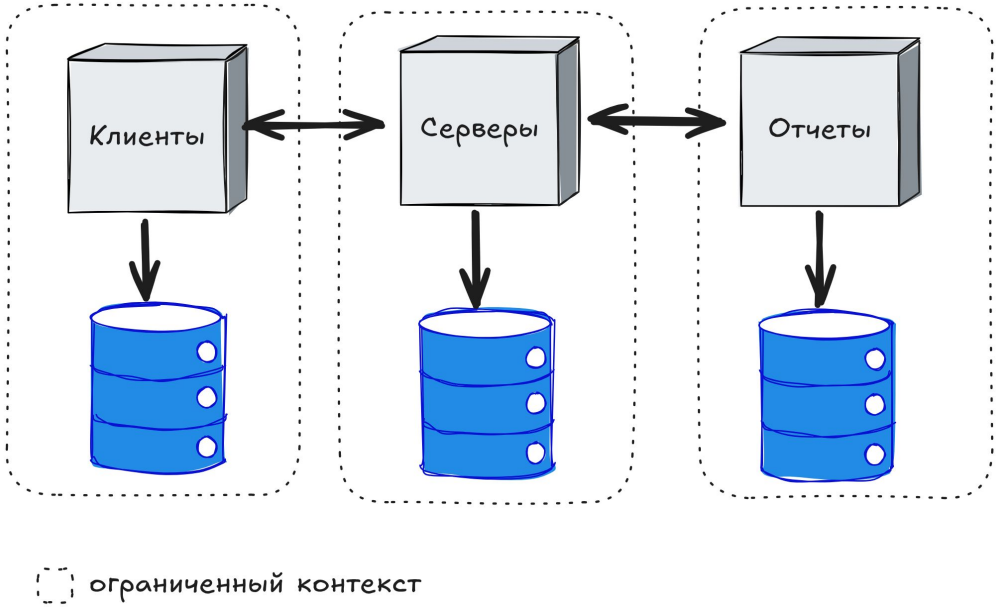
Sidecar

- Выполнение периферийных задач
  - мониторинг
  - безопасность
  - отказоустойчивость
- Отдельный компонент
  - может быть на другом языке



Database per Service

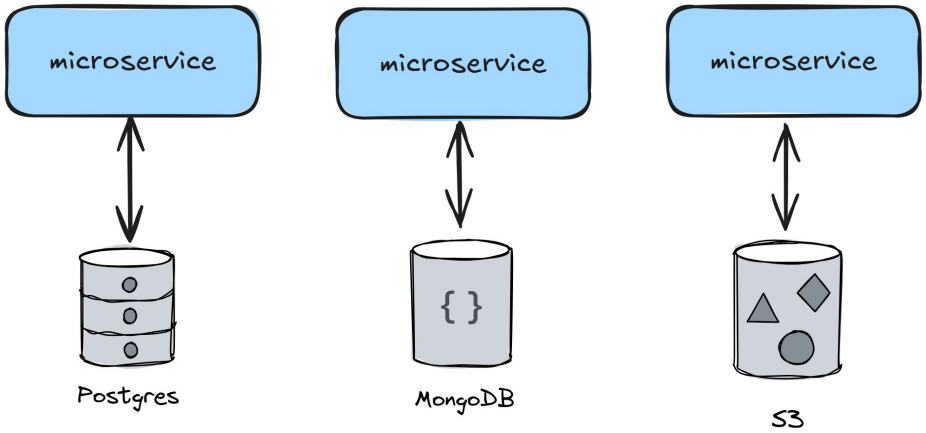
- Микросервис владеет своими данными (БД на микросервис) - ограниченный контекст
- Основная причина - управление изменениями / их изоляция



Polyglot Persistence

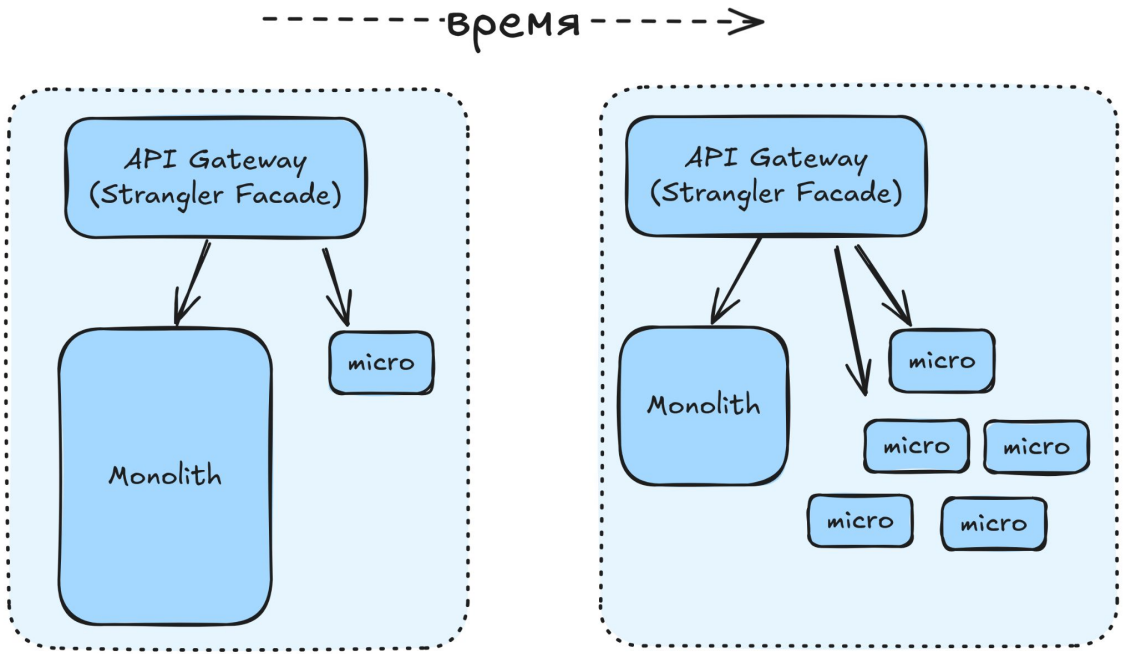
«Правильный инструмент для правильной задачи»

- Несколько технологий хранения данных
  - Реляционные БД для таблиц
  - Документо-ориентированные БД для документов
  - Key-value для кэша и быстрых ключей
  - Колоночные БД для аналитики
- Локальность данных
- Независимость масштабирования
- Явные границы ответственности



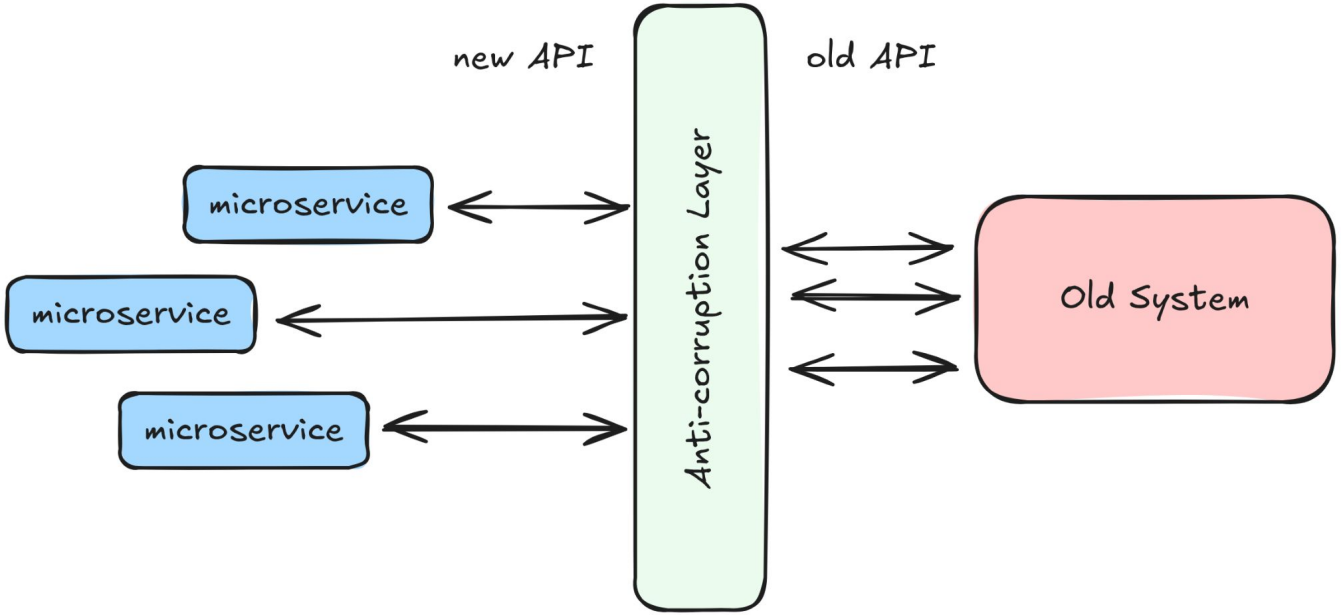
Strangler

- Миграция монолита на МСА
- Постепенный перенос функций
- Маршрутизация запросов между монолитом и микросервисами



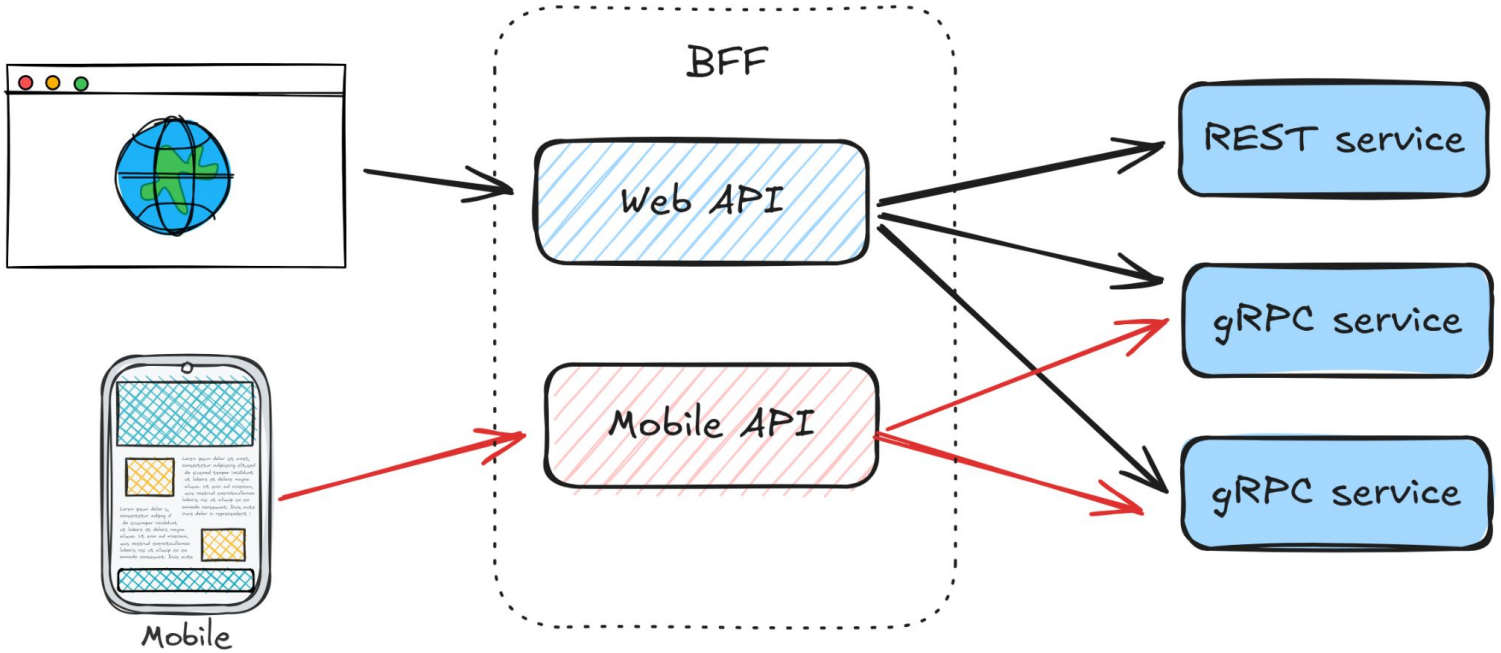
Anti-Corruption Layer

Защита от влияния внешних систем с несовместимой или «загрязненной» логикой, например, старых монолитов



Backends for Frontends (BFF)

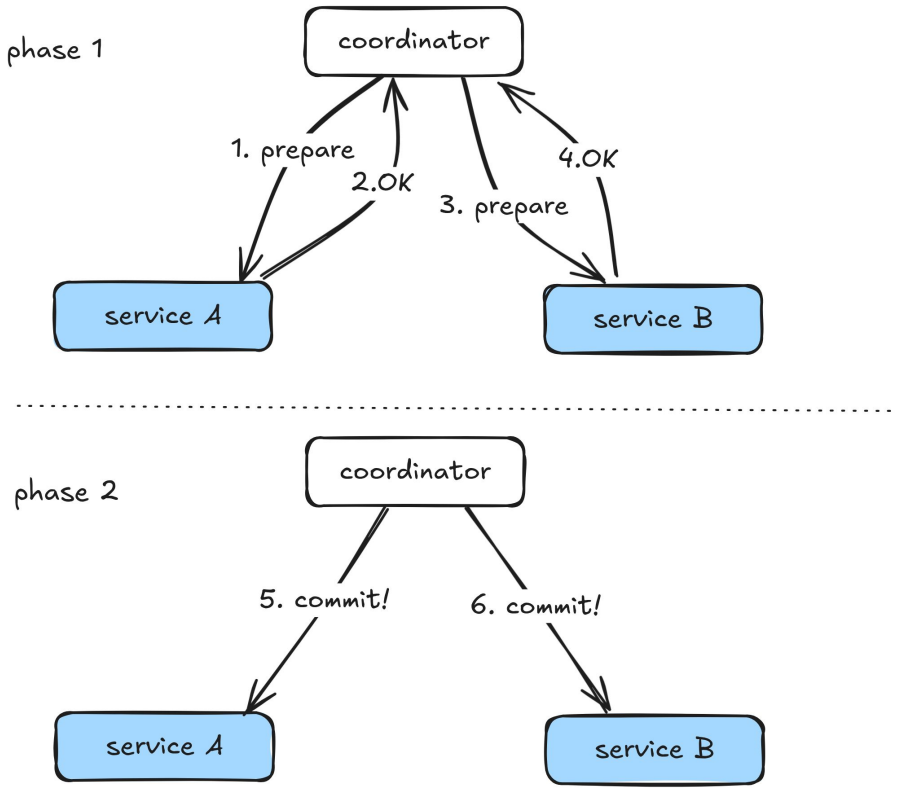
Несколько API Gateway, адаптированных под клиентов



2-phase commit (not msa pattern)

Распределенная транзакция, 2 фазы

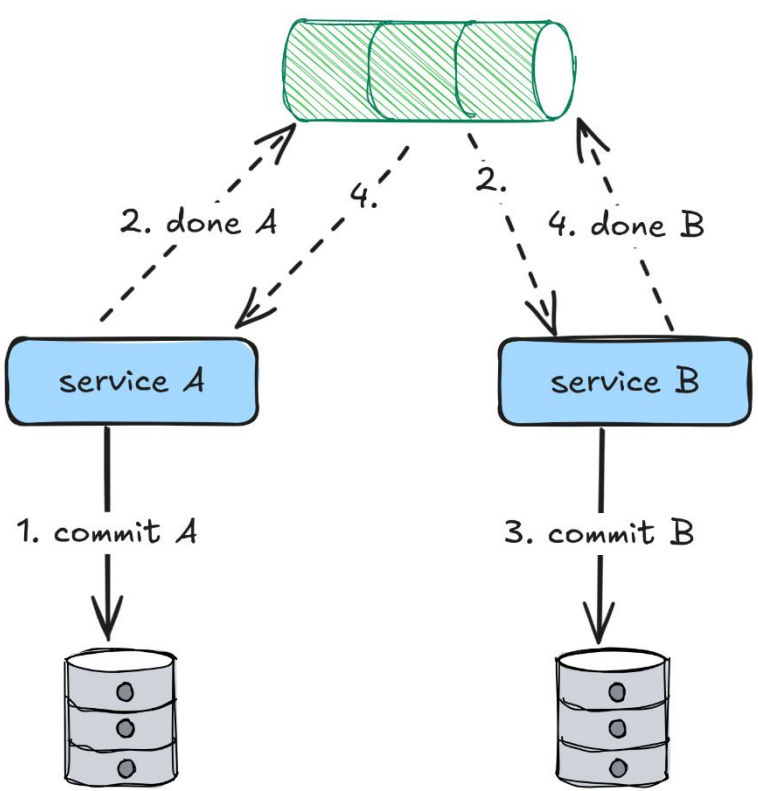
- Все готовы к изменениям?
- Всем запомнить изменения!

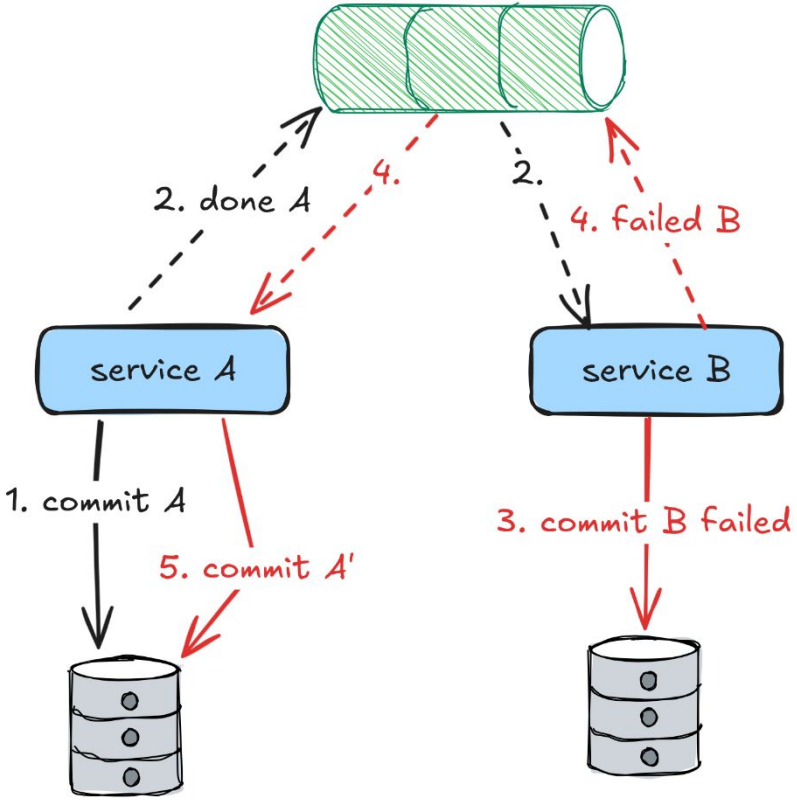


Saga

- Координация многошаговых бизнес-процессов, охватывающих несколько микросервисов
- Процесс может быть длительным
- Обеспечивает согласованность данных без классических ACID-транзакций
- Локальные транзакции и отмены
- Хореография или оркестрация коммитов

Saga

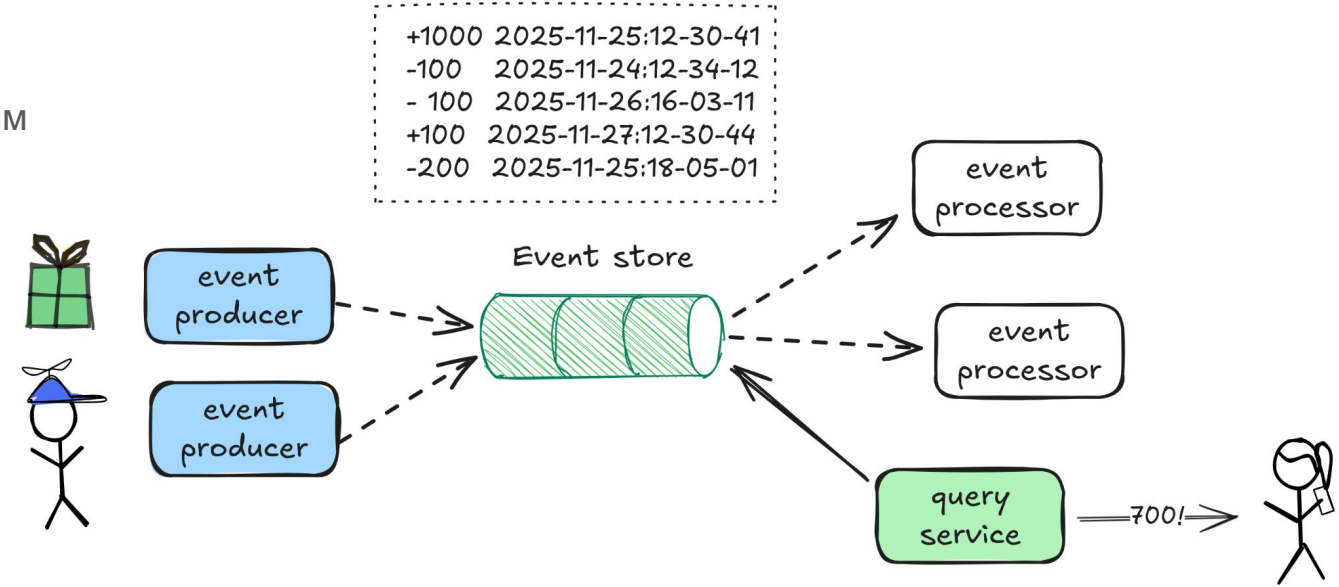

2. done A
4.
2.
4. done B
service A
service B
1. commit A
3. commit B


2. done A
4.
2.
4. failed B
service A
service B
1. commit A
5. commit A'
3. commit B failed

Event Sourcing

Состояние системы сохраняется как последовательность неизменяемых событий, описывающих все произошедшие изменения.

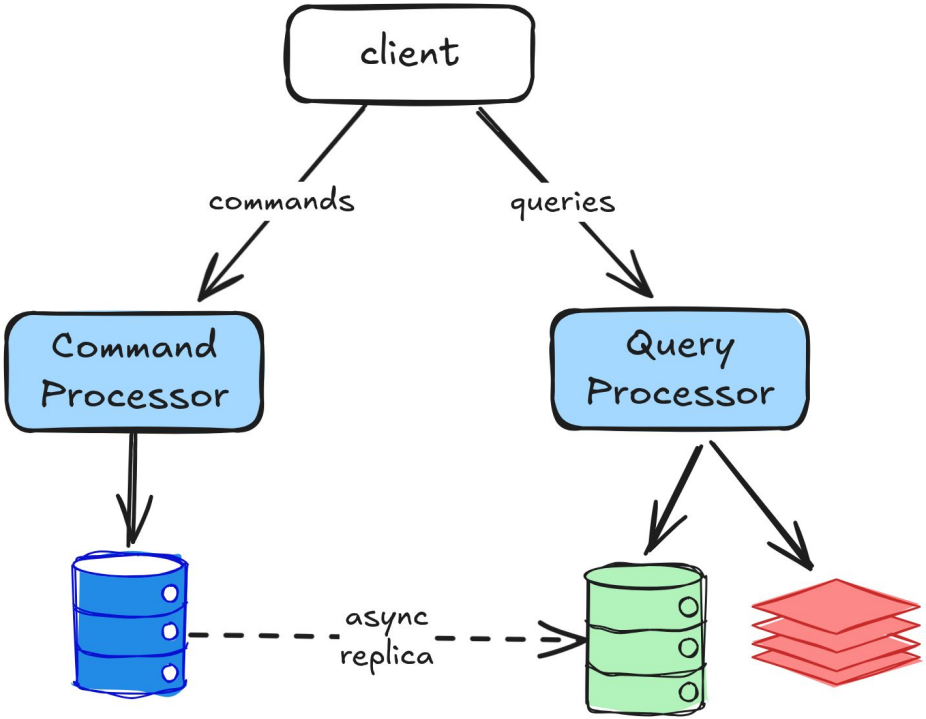
- Полная история изменений
- Аудит
- Масштабируемость
- Устойчивость к сбоям



CQRS

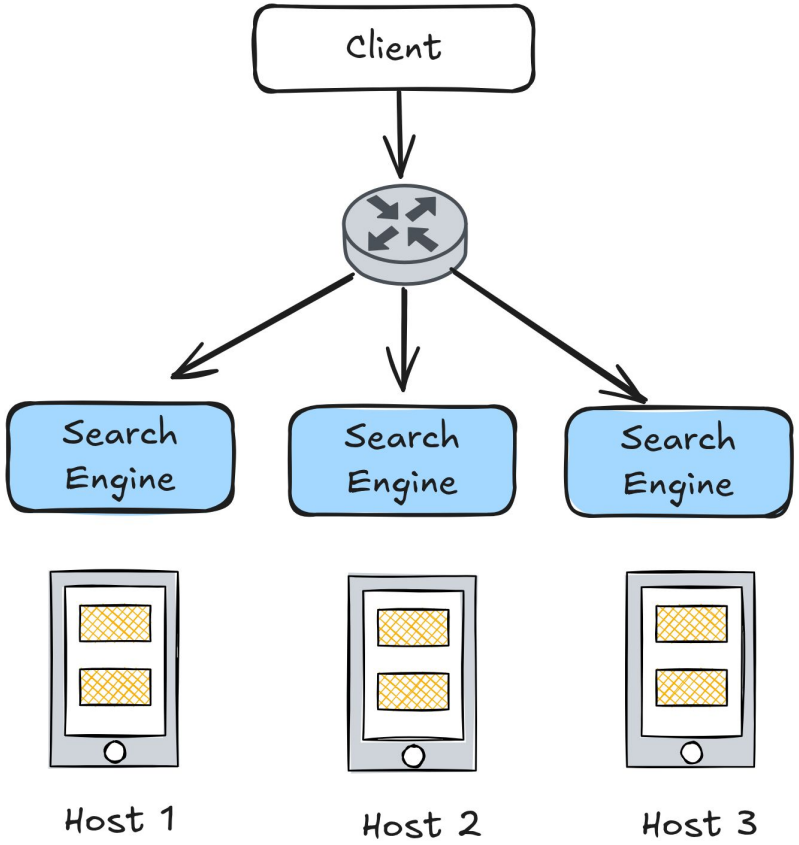
Разделение операции чтения и записи на разные модели данных (базы)

- Command-модель обрабатывает изменения (создание, обновление, удаление)
- Query-модель оптимизирована для чтения (выборка, фильтрация, агрегация, денормализация)
- Масштабируемость, производительность



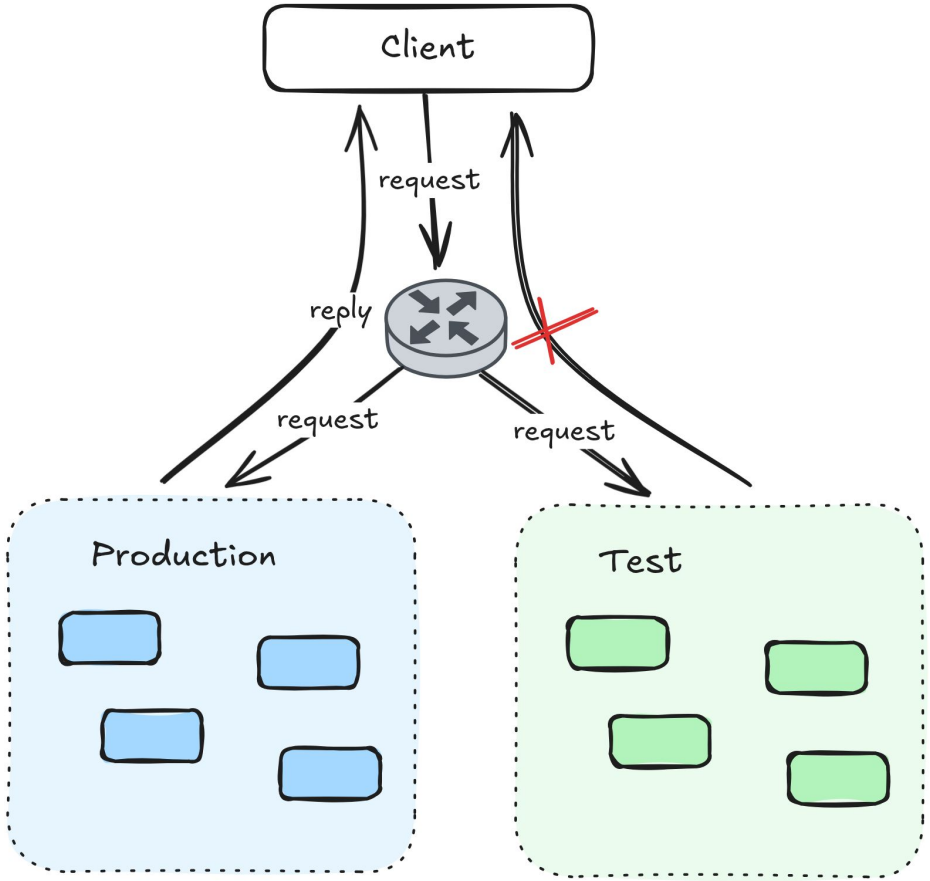
Service Instance Per Host

- Изоляция ресурсов - экземпляр получает всю мощность хоста
- Упрощенное масштабирование
- Предсказуемая производительность
- Простота мониторинга
- Высокая доступность



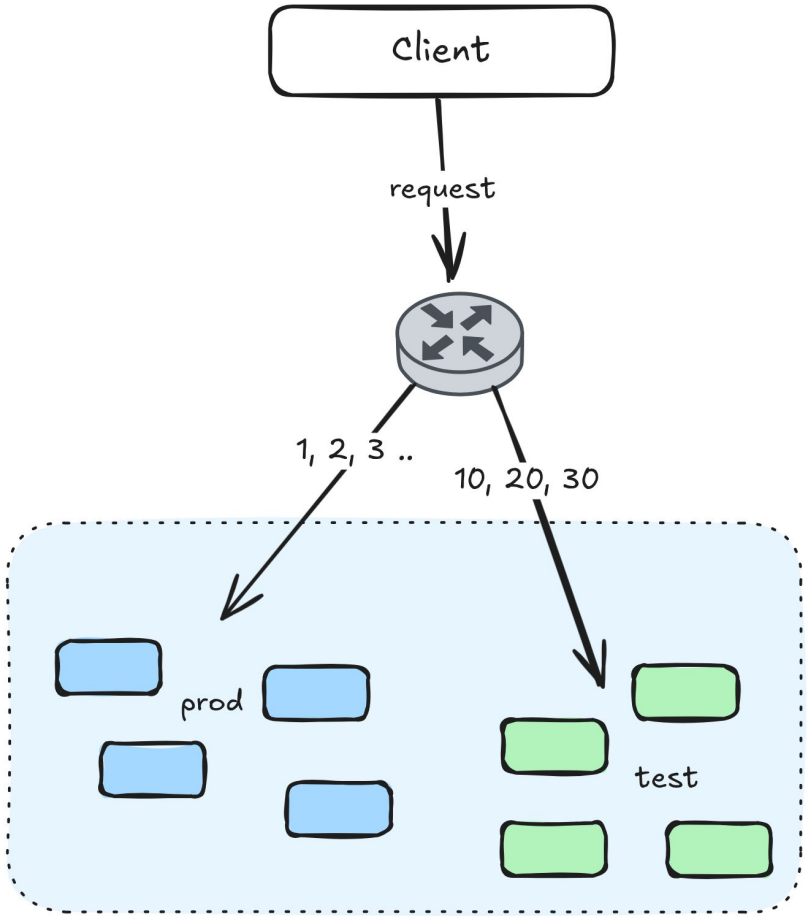
Shadow Deployment

- Новая версия приложения запускается параллельно с действующей версией
- Реальный трафик дублируется и направляется в обе системы
- Ответы от теневой версии не показываются пользователям
- Тестирование, сравнение



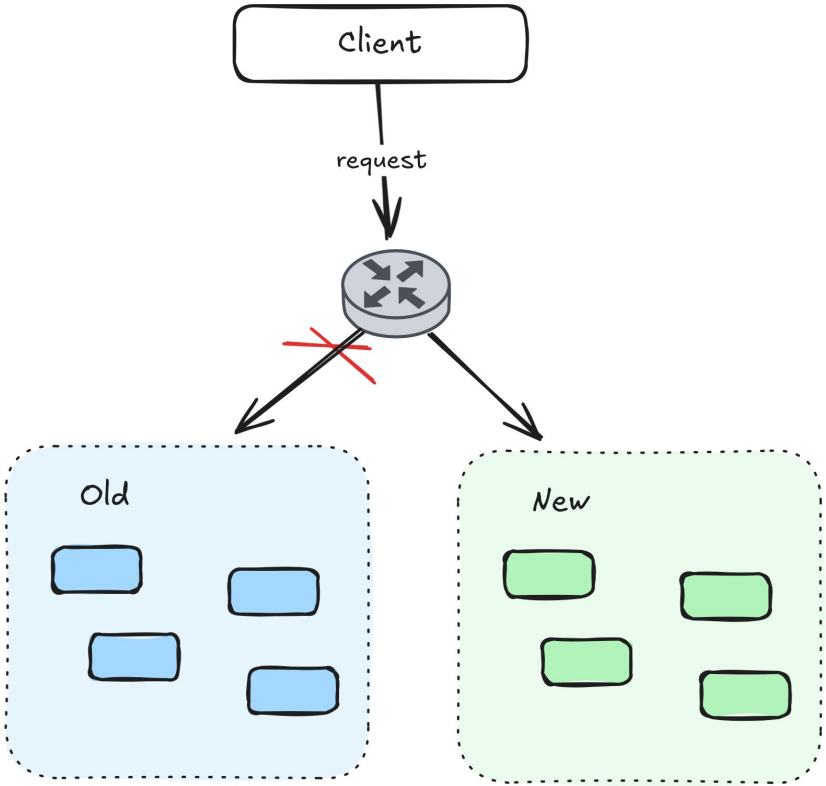
Canary Deployment

- Часть трафика идет на новые (тестовые) версии
- Пользователи также видят результаты с новых микросервисов
- Постепенное увеличение доли трафика



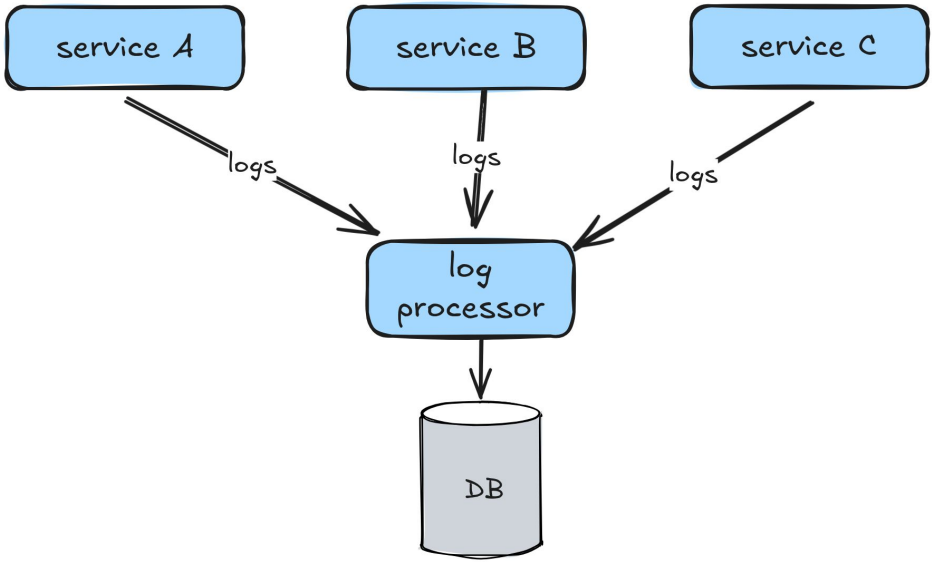
Blue-Green Deployment

- Одновременно развернуты старая и новая версии
- Только одна версия работает
- Переключение разом



Log Aggregation

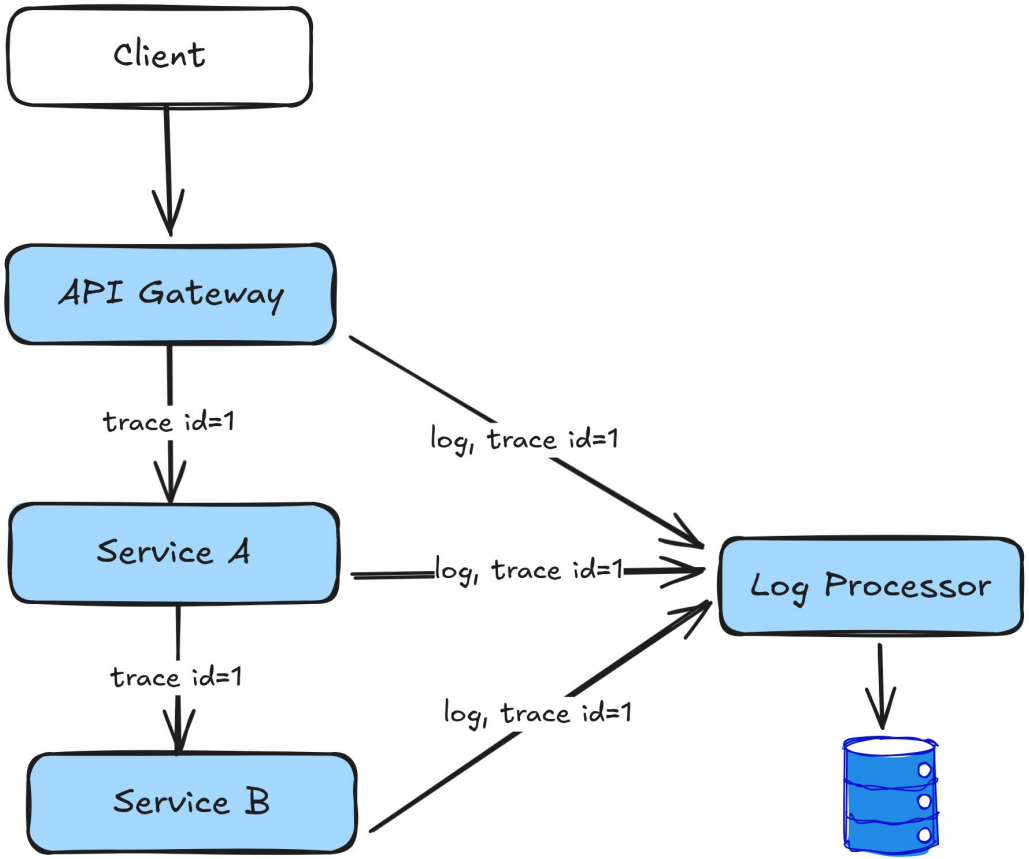
Сбор, централизованное хранение и обработка логов в едином репозитории для упрощения анализа и мониторинга



Distributed Tracing

Мониторинг запросов в распределенных системах

- Маршрут
- Выявление узких мест
- Ошибки



Место под ваш любимый паттерн

Помним про компромисы!

У каждого паттерна есть

- Задача
- Область применения
- Сильные и слабые стороны

